

Token Factory: Efficiently Integrating Diverse Signals into Large Recommendation Models

Xilun Chen*
xilunchen@google.com
Google
USA

Lukasz Heldt
heldt@google.com
Google
USA

Aniruddh Nath
aniruddhnath@google.com
Google
USA

Shao-Chuan Wang*
scwang@google.com
Google
USA

Lichan Hong
lichan@google.com
Google
USA

Li Wei
liwe@google.com
Google
USA

Baykal Cakici
baykal@google.com
Google
USA

Raghu Keshavan
hkraghunandan@google.com
Google
USA

Xinyang Yi
xinyang@google.com
Google
USA

Abstract

Large Recommendation Models (LRMs) have demonstrated promising capabilities in industry-scale recommendation tasks. However, holistically integrating traditional signals into these transformer-based architectures effectively and efficiently remains a major challenge. Conventional approaches that "textualize" these signals directly or create discrete item representations often lead to excessively long prompts, substantial memory footprints, and high computational overhead. To overcome these limitations, we propose "Token Factory", a framework designed to transform traditional signals into "soft tokens" that can be directly processed by LRMs. This approach enables efficient integration and compression of heterogeneous input features, preventing prompt length explosion while enhancing model performance. We detail the architecture of Token Factory and present experimental results validating its effectiveness in a production-scale recommendation environment.

ACM Reference Format:

Xilun Chen, Shao-Chuan Wang, Baykal Cakici, Lukasz Heldt, Lichan Hong, Raghu Keshavan, Aniruddh Nath, Li Wei, and Xinyang Yi. 2026. Token Factory: Efficiently Integrating Diverse Signals into Large Recommendation Models. In *20th ACM Conference on Recommender Systems (RecSys '26)*, September 27–October 02, 2026, Minneapolis, MN, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3773078.3831865>

1 Introduction

In modern industrial recommender systems, Large Recommendation Models (LRMs) based on Transformer architectures have demonstrated great promise [3–6]. These models typically rely on a diverse array of signals and advanced processing techniques,

including Semantic ID generation [12], custom tokenization, continual pre-training (CPT) [16], task-specific fine-tuning [4] and user modeling [8]. A representative task in this domain is next-video prediction, as illustrated in Figure 1.

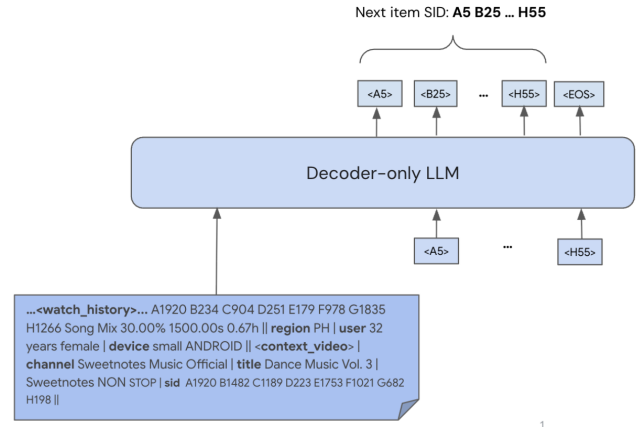


Figure 1: Illustration of Generative Retrieval for next video recommendation from [4]. The input prompt is a sequence of interleaved SID tokens, text, and custom tokens for numerical features.

Traditional signals, such as the dense and sparse features that form the foundation of Large Embedding Models (LEMs), carry vital information. They encapsulate direct user interactions and domain-specific insights engineered to effectively guide the recommendation process. However, a primary challenge lies in the efficient representation of these traditional signals within Transformer architectures. Directly mapping these heterogeneous signals to text often results in prohibitive prompt lengths, excessive memory overhead, and high computational costs. For instance, in the next-video prediction task, converting dense signals (such as watch completion ratios and watch times) into text for processing via a custom tokenizer creates a significant computational bottleneck.

*Equal contribution. Alphabetical order.



This work is licensed under a Creative Commons Attribution 4.0 International License. *RecSys '26, Minneapolis, MN, USA*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2284-4/2026/09
<https://doi.org/10.1145/3773078.3831865>

To address this challenge, we introduce **Token Factory**, a framework designed to transform traditional signals into *soft tokens* that serve as a new modality for Large Recommendation Models. A "soft token" is defined as an embedding representation directly within the multi-modal model's embedding space, rather than a discrete ID derived from a fixed vocabulary. Unlike standard "hard" tokens that require a tokenizer to map text to vocabulary indices, soft tokens are generated directly from raw feature values through learnable embedding tables and transformations. These soft tokens are critical because they enable the model to ingest high-dimensional, continuous, and non-textual signals as a new modality without inflating the prompt length, thereby preserving precise information that might be lost or inefficiently represented during conversion to text.

Our core contributions are summarized as follows:

- We demonstrate the critical importance of leveraging a rich set of traditional signals in modern Transformer-based Large Recommendation Models (LRMs), such as PLUM [4], through extensive experimental analysis, thereby addressing the limitations of current state-of-the-art methods that often restrict these signals due to computational trade-offs.
- We introduce **Token Factory**, a novel architectural framework that enables the effective and efficient mapping of heterogeneous traditional features into "soft token" representations, serving as a new modality for recommendation models.
- We validate the empirical effectiveness of Token Factory through offline and online evaluations, including ablation studies, demonstrating that it achieves on-par or superior quality in ranking and retrieval while enabling significant efficiency improvements.

The remainder of this paper is organized as follows. We will first discuss related work and present the proposed Token Factory framework. We then share our experimental results and future work.

2 Related Work

Large Recommendation Models. Transformer-based Large Recommendation Models have gained tremendous attention and success in the industry over the past years. Extensive research has focused on effective input representation and model architecture optimization. For example, HSTU [13] scales via architectural enhancements and optimization; PLUM [4] focuses on leveraging Semantic IDs to represent high-dimensional items for the generative recommendation task; and SEATER [11] proposes generating item identifiers that capture the intrinsic semantics of items as indices, rather than treating them merely as text inputs. TokenRec [10] proposed a novel ID tokenization strategy that quantizes masked user/item representations into discrete tokens to incorporate high-order collaborative knowledge. While these approaches excel at modeling user sequences or optimizing architectures, they still rely on textualizing auxiliary features. In contrast, Token Factory provides a unified framework to map heterogeneous signals directly into the Transformer's embedding space as soft tokens, maintaining prompt compactness while preserving feature richness.

The concept of using soft tokens in LRMs is related to CoLLM [15], which treats collaborative features as a distinct modality and maps pre-trained embeddings into the LLM's token space via an MLP. While CoLLM is restricted to user and item collaborative embeddings, Token Factory generalizes this approach to accommodate a highly heterogeneous mix of dense, sparse, and sequential signals. Other recent frameworks, such as LLM4Rec [9], integrate diverse modalities (e.g., text, video, and audio) using specialized encoders and cross-modal attention. While powerful, such heavy fusion architectures typically introduce significant computational complexity and prompt-length overhead. Conversely, Token Factory was designed with strict efficiency constraints in mind to ensure compatibility with latency-sensitive industrial environments.

Token Compression. To further mitigate the scaling challenges when considering long user interaction history, several methods have been proposed to "shorten" or "compress" the input tokens. For example, LONGER [2] optimized representations by introducing a global token and a token merge strategy to merge every K items. GenRec [20] used a linear projection to encode long user behavior sequences with multi-token item representations based on semantic IDs; however, it does not consider the other features associated with each item, such as user-item interactions. HyMiRec [18] converted items (title + content) into "SID" like codes (3 per item) to reduce input token lengths. Other approaches include segmenting sequences into chunks with learnable tokens and compressing historical segments into personalized expert tokens [14], or using hierarchical K-means clustering [17]. Existing compression techniques primarily focus on reducing the number of item IDs in a sequence, often ignoring the rich interaction features associated with each item and solely depending on SIDs or item embeddings. Token Factory allows these features to be fused into the item's soft token representation during compression, ensuring that the compressed representation retains critical information.

3 Token Factory

In this section, we present our proposed Token Factory in greater detail.

3.1 Overview

Figure 2 illustrates the overall architecture of Token Factory. Token Factory consists of a collection of **Token Makers**, each of which produces a sequence of "soft tokens". The traditional signals such as dense, sparse, and embedding features are fed into various token makers and the produced "soft tokens" are then placed at the desired positions in the prompt for the LRM, as shown in Figure 2.

Specifically, within a video recommendation system scenario, the exemplary Token Factory architecture consists of three token makers, namely

- **WH Token Maker**, which is responsible for producing soft tokens from the user interaction sequence (e.g., videos in the user watch history).
- **Query Token Maker**, which is responsible for leveraging and integrating query-level features (user features).
- **Candidate Token Maker**, which is responsible for leveraging and integrating candidate-level features (video features or any non-query level features).

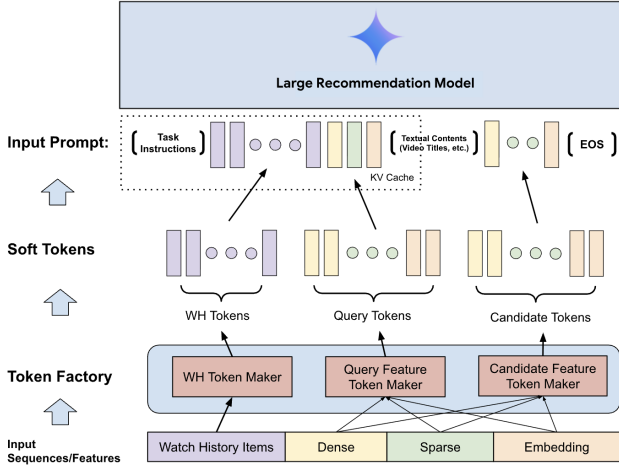


Figure 2: Token Factory Architecture. Traditional signals are transformed into "soft tokens" as another modality and integrated with textual tokens in the prompt.

Notably, these soft tokens can be seamlessly interleaved with textual tokens and benefit from prefix caching. Because query- and user-level features remain static while scoring multiple candidate items within the same recommendation request, their corresponding soft tokens can be pre-computed and cached to significantly reduce inference latency.

This architecture offers several key benefits:

- **Prompt Compactness:** The prompt length can be significantly reduced compared to directly "textualizing" the traditional signals via custom tokenization.
- **Deterministic Length:** The prompt length can be deterministically capped at a predefined budget, irrespective of the number of input features. This is achieved because each Token Maker uses a fixed-output projection layer to map an arbitrary number of concatenated features into a strict, predetermined number of soft tokens.
- **Longer User Interaction:** It enables the consideration of much longer user interaction sequences through compression of multiple signals into one (or fewer) soft token.
- **Efficiency:** It facilitates more efficient training and serving due to the compacted prompt.

To realize these architectural benefits, the core component responsible for feature transformation is the Token Maker. In the following section, we provide a detailed exposition of its design and operation.

3.2 Token Maker

A Token Maker consists of a set of input features and a specification for the target output token(s), encapsulating the logic required to transform these features into the desired number of soft tokens.

To be more specific, let's assume $F_{input} = [f_1; f_2; \dots; f_n]$, where f_i indicates the i -th raw input feature to the Token Maker. Based on the characteristics of the feature, further normalization or embedding lookup could be applied. Therefore, the final features passed

to the Token Maker are the concatenation of all transformed features as $E_{input} = \text{Concat}(t_1(f_1), t_2(f_2), \dots, t_n(f_n))$, where t_i is the transformation function to use for feature f_i .

We then define a function that further transforms E_{input} into soft token(s) by

$$T_{output} = G(E_{input})$$

where T_{output} is the N output soft token(s), each of dimension d_{model} , which makes them suitable to be directly consumed as embeddings by the Transformer. The function G is a differentiable mapping that transforms E_{input} into an $N \times d_{model}$ embedding vector, which is ultimately reshaped into N soft tokens. Crucially, G is co-trained end-to-end with the Large Recommendation Model, ensuring that the generated soft tokens are aligned with the LLM's semantic space and task objective. The function G can be as simple as a Multi-Layer Perceptron (MLP), or as complex as a neural network or a transformer.

The choice of N balances feature capacity against computational latency. A larger N preserves more high-dimensional information but increases the downstream Transformer's sequence length and attention overhead. In practice, N is determined empirically based on the feature type: simple features are mapped to a single token ($N = 1$), while long interaction sequences are compressed into a small, fixed budget (e.g., $N = 10$) to optimize serving efficiency.

Figure 3 illustrates the core architecture of the Token Maker, which projects all features associated with a single watch history item into a unified soft token representation. In this example, an individual watch item comprises heterogeneous features—including the video Semantic ID (SID), channel name, client information, and watch duration. These features are concatenated and processed through a Multi-Layer Perceptron (MLP) layer to map them into the final soft token embedding space. Because the user history is inherently sequential, sequentially applying this projection transforms the entire watch history into a corresponding sequence of soft tokens within the model input.

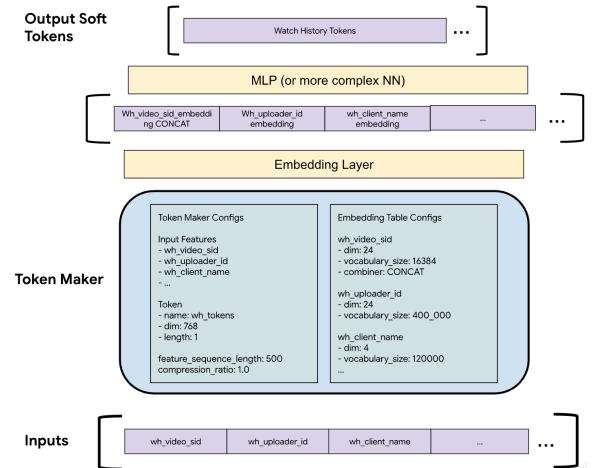


Figure 3: Token maker for user watch history.

Due to the flexibility and extensibility of the Token Maker module, one can naturally leverage ideas from many state-of-the-art token generation approaches, such as [7, 19].

3.3 Evolution of Prompts

To visualize the changes in the prompt, we compare the prompt representations before and after applying Token Factory.

Textual Semantic ID Prompt: In previous work such as PLUM, features like SID were textualized. For example, a user’s watch history item might be represented by its SID and associated metadata. An 8-token SID would consume 8 tokens in the prompt. A typical prompt template used in Figure 1 might look like:

```
[watch_history] | region [user_region] | user [user_age]
[user_gender] device [device_info] | video [video_title]...
```

The actual prompt would expand to:

```
A100 B100 C100 D100 E100 F100 G100 H100 5.00% 50.00s 2.5h | ...
region PH | user 32 years female | device small ANDROID | video
baby shark...
```

where A100 to H100 are the 8 tokens for SID, together with a few dense features, i.e., watch completion ratio, video watch time, and time since watch, that are repeated across the length of the actual watch history. This results in very long prompts for even moderate watch history lengths.

Token Factory Prompt: With Token Factory, the traditional features (dense, sparse, sequence) are transformed into soft tokens. The prompt length is significantly reduced. The soft tokens (indicated by `< emb >` below) do not have a human-readable text representation as they are embedding vectors, but conceptually the prompt would look like this:

```
[watch_history]<emb><emb><emb>...<emb>[end_watch_history]
<emb>...<emb> video baby shark <emb><emb>...<emb>
```

In this example, each `< emb >` token between `[watch_history]` and `[end_watch_history]` corresponds to a single watch history item, replacing the verbose textual representation (e.g., 'A100 ... 2.5h'). The number of these `< emb >` tokens matches the history length. We retain some textual tokens, such as the video title, to leverage the LLM’s natural language understanding capability. The additional `< emb >` tokens following the title are generated from other heterogeneous features of the user, candidate item, and their interactions.

With this transition, items are no longer represented by discrete sequences of textual SID tokens. Consequently, the individual sub-tokens associated with different watch history items do not participate in direct, token-level attention. To verify that the model still successfully captures interactions across watch history items despite the absence of explicit SID tokens, we present a comparative analysis of the attention weights for both soft tokens and textual SIDs in Appendix A. This analysis demonstrates that the compressed soft tokens continue to facilitate sufficient and effective cross-item attention within the interaction sequence.

3.4 Sequence Compression

In Section 3.2, we showed an example of how one watch history item transforms into one soft token via the Token Maker. However, this may not be sufficiently efficient when dealing with longer

watch histories (e.g., if we want to encode thousands of watches or even a user’s lifetime watches). Therefore, we need to further compress the sequence into fewer soft tokens.

3.4.1 MLP Compression. One approach to further compression is using an MLP on the sequence dimension. Assume that the Token Maker has already converted the watch history to soft tokens, where one item is represented by one token. This results in a JAX [1] array of shape $[batch_size, sequence_length, token_dim]$, where $sequence_length = N$. An MLP layer of shape $[N, \dots, M]$ can be applied on the second dimension to reduce the sequence length from N to M . As a result, we end up with M soft tokens representing the entire sequence, achieving a compression ratio of M/N .

3.4.2 Attention Pooling. Inspired by [2], we could also apply a lightweight transformer and use attention pooling on the last item in every K items in the sequence. This essentially summarizes every K items using one soft token and results in a compression ratio of $1/K$.

4 Experiments

In this section, we conduct a comprehensive set of experiments on various use cases within our Large Recommendation Model PLUM framework, including ranking and retrieval tasks, to validate the proposed Token Factory framework. We first evaluate the performance of the PLUM-based ranking model and the PLUM-based generative retrieval model to demonstrate the efficiency and effectiveness of Token Factory. We then perform various ablation studies to understand the importance of traditional signals and Semantic IDs that are used in Token Factory, as well as the impacts of user interaction history length.

4.1 Results on Ranking Task

In this section, we detail our experimental results for PLUM with Token Factory in the ranking task (i.e., predicting the CTR of a given video).

4.1.1 Experiment Setup. We run the experiment on a PLUM model derived from 110M Mixture-of-Experts (MoE) variant of Gemini encoder model. Training data consists of user watch history, traditional dense and sparse features, and the clicks for the next video prediction task.

- **Baseline:** The baseline uses the textual Semantic ID and custom tokenization of several dense signals using the same PLUM model. In the baseline prompt, each watch history item takes 12 tokens (8 tokens for the SID, 1 token for the channel name, and 3 tokens for dense features in textual format). We use a prompt length of 1536 tokens as input, which contains 200 watch history items, user demographic information, and the candidate video title in textual format. Excessive tokens beyond the 1536 limit are truncated from the left, which specifically discards the oldest watch history items to preserve the user’s most recent interactions and target metadata.
- **Treatment:** The treatment uses Token Factory to transform each watch history item into one soft token. As a result, the prompt length is significantly reduced to 480 tokens.

4.1.2 Experiment Results. The primary research question we aim to answer in this evaluation is: *Can Token Factory achieve on-par predictive quality compared to the baseline while operating on a significantly compressed prompt?* Figure 4 addresses this by showing the comparison between the baseline and the treatment on ROC AUC for the CTR prediction task. Both models use the same global batch size even though the treatment has a much shorter input length. We observe that the model using Token Factory that converts textual watch items into soft tokens tends to have a lower AUC initially, but catches up to the baseline at 1.5M steps and consistently performs comparably afterwards. This initial lag is expected because Token Factory introduces additional randomly initialized parameters in the token makers and new embedding tables for sparse features. These components must learn from scratch how to effectively project the raw signals into the LLM’s semantic space. In contrast, the baseline maps features to text, which may be easier for the model to optimize initially, albeit less efficiently. Once the token makers are sufficiently trained, the treatment catches up and matches the baseline’s quality, demonstrating that the soft token representation is as effective as the full textual representation while being much more compact.

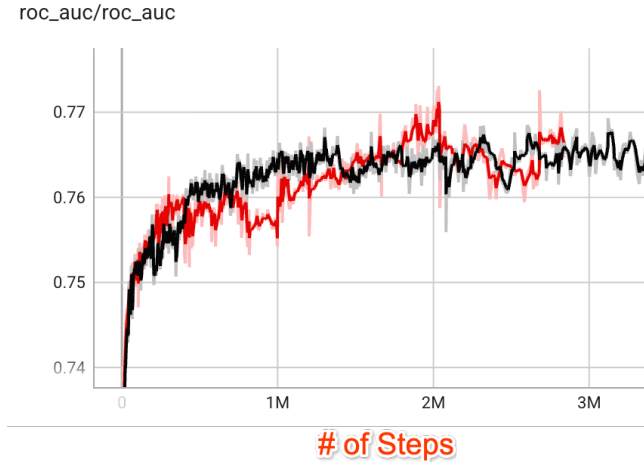


Figure 4: ROC AUC comparison between baseline (black line) and treatment (red line), keeping the same batch size.

With this setup, we can also achieve a significant improvement in training speed of about 200% faster due to the fact that the prompt length is only about 30% of the baseline length.

Figure 5 shows the comparison between the baseline and the treatment on ROC AUC for the CTR prediction task when we increase the global training batch size for the treatment by 200%. We observe that now the model with Token Factory is outperforming the baseline even at a very early stage due to the increased batch size. The efficiency gain from Token Factory unlocks the consideration of much bigger batch size and improves the overall performance of the model.

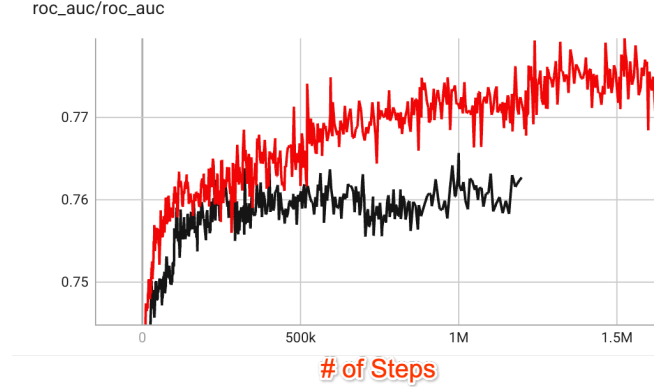


Figure 5: ROC AUC comparison between baseline (black line) and treatment (red line), with increased batch size.

4.2 Results on Generative Retrieval Task

In this section, we detail our experimental results for PLUM with Token Factory in the generative retrieval task (i.e., predicting the Semantic IDs for the next video to watch).

4.2.1 Experiment Setup. We run the experiment on a PLUM model derived from 210M Mixture-of-Experts (MoE) variant of Gemini decoder model. Training data consists of user watch history, traditional dense features, and the semantic IDs for the next video as the target.

- **Baseline:** The baseline relies on textual Semantic IDs and custom vocabulary to represent dense signals. In this setup, each watch history item consumes 5 tokens (1 token for a random hash of the SID sequence representing an item, 1 token for a random hash of the channel name, and 3 tokens for dense features in textual format). The total prompt length is capped at 768 tokens, which includes the user’s demographic information in textual format.
- **Treatment:** The treatment uses Token Factory to transform each watch history item into one soft token. The prompt length is 256, which includes 200 watch history items. The user demographic features used in textual format in the baseline are also converted into soft tokens.

4.2.2 Experiment Results. We use recall@10 to measure model’s offline performance and also report our online experiment results in production. For online performance, we use “Unique Impressions,” which measures the candidates that are uniquely generated by the model and shown to the user, and Satisfied Watchers, and Satisfied Watch Time to measure user engagement of the new model.

Table 1 presents both offline and online evaluation results for the generative retrieval task on YouTube Homepage recommendation. As shown in the table, the model using Token Factory achieves a +2.0% improvement in offline Recall@10 compared to the baseline. In online experiments, we observed a significant improvement on Unique Impressions (increased by +16.8%), and notably, Unique Impressions for one-day-fresh videos soared by 67.1%. This demonstrates that Token Factory enables the LRM to effectively leverage holistic signals from features and efficiently consider longer user

Offline	Online			
Recall@10	Unique Impressions	Unique Impressions (1d fresh)	Satisfied Watchers	Satisfied Watch Time
+2.0%	+16.8%	+67.1%	+0.04%	+0.05%

Table 1: Offline and Live Experiments for using Token Factory in a PLUM based model in the retrieval stage for YouTube Homepage recommendation. The new model is much more effective for new videos, as measured by the Unique Impressions of one-day-fresh videos.

interaction sequence, greatly enhancing its ability to retrieve and recommend fresh content. Furthermore, these improvements in exploration did not come at the cost of user satisfaction; we achieved positive gains in both Satisfied Watchers (+0.04%) and Satisfied Watch Time (+0.05%).

4.3 Scaling Studies

In this section, we investigate the following research question: *Does Token Factory primarily serve as an efficiency optimization, or does the soft token representation also provide headroom for quality improvements?* To answer this, we designed four ablation studies to isolate the factors impacting soft token performance in the ranking task. All treatments and the baseline use a fixed prompt length of 480 tokens (which contains 200 watch history items, the candidate video SID, the video title, and associated traditional dense and sparse features) with a batch size of 32k. When the actual prompt exceeds this length, it is truncated from the left.

- **Baseline:** Same setup as the treatment in Section 4.1.1.
- **WH_SID:** Similar to Baseline but uses the textual SID format for watch history as oppose to using soft tokens for watch history
- **NO_FEAT:** Similar to Baseline but removes all traditional dense and sparse features (except those from the watch history).
- **WH_SID_NO_FEAT:** Similar to **WH_SID** but removes all traditional dense and sparse features.
- **NO_FEAT_STRICT:** Similar to **NO_FEAT**, but further removes the dense and sparse features from the watch history as well. With this setup, a soft token for watch history only contains the SID and channel ID.

Figure 6 shows the AUC comparisons and we observe that:

- Using soft tokens or the SID format did not show a clear gap when all dense and sparse features are present.
- Comparing **NO_FEAT_STRICT** with **WH_SID_NO_FEAT**, we see that using soft tokens for the watch history performs better than using SIDs in textual format. This is mostly due to the context window budget or constraint (480) that we set: the soft token models were able to include 200 watch history items into the prompt, whereas SIDs may get truncated due to budget constraints.
- When comparing **NO_FEAT** and **NO_FEAT_STRICT**, we can conclude that adding more features in each soft token still helps with the CTR prediction task.

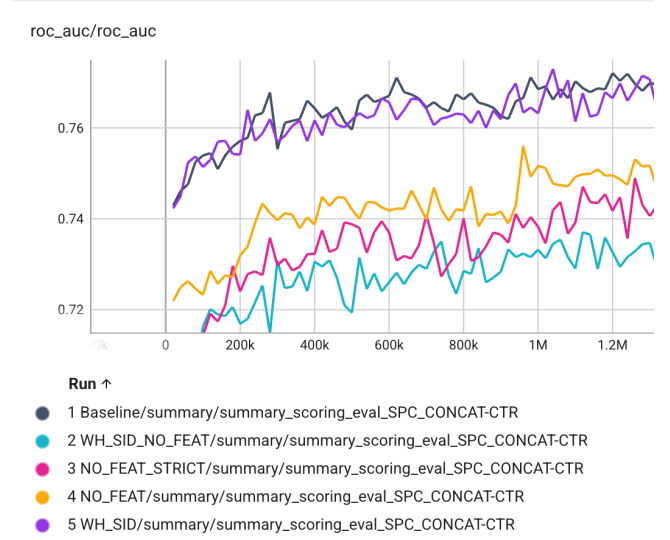


Figure 6: ROC AUC comparison.

4.3.1 Longer Watch History. With the compression capabilities described in Section 3.4, we further conducted a study to extend the watch history from 200 to 500 in the CTR prediction task and use a compression ratio of 10% to compress the 300 added watch history items into 30 soft tokens. We observe a +0.08% improvement in AUC. Note that in this study, the treatment has an increased prompt length of 30 tokens compared to baseline to hold the extra compressed watch history soft tokens.

5 Conclusion

In this paper, we present Token Factory, a novel framework designed to efficiently integrate diverse traditional signals into modern Transformer-based Large Recommendation Models. By transforming heterogeneous features such as dense, sparse, and sequence signals into "soft tokens," our approach successfully circumvents the challenge of prompt length explosion that arises from direct textualization. Our experimental results on both ranking and generative retrieval tasks in an industrial-scale recommendation system demonstrate that Token Factory not only significantly reduces training and serving costs but also maintains or improves model quality.

Acknowledgments

We would like to thank Min-hsuan Tsai for providing valuable feedback to this paper, and the following individuals for applying Token Factory to various surfaces and use cases (alphabetical order): Dapeng Hong, Vikas Kedigehalli, Yuxuan Li, Sunny Liu, Yang Liu, Steven Su, Yueqi Wang, Bo Yan, Anlan Yu, Qiao Zhang, Rein Zhang, and Zelong Zhao.

References

- [1] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. *JAX: composable transformations of Python+NumPy programs*. <http://github.com/google/jax>

- [2] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, et al. 2025. Longer: Scaling up long sequence modeling in industrial recommenders. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 247–256.
- [3] Sunhao Dai, Jiakai Tang, Jiahua Wu, Kun Wang, Yuxuan Zhu, Bingjun Chen, Bangyang Hong, Yu Zhao, Cong Fu, Kangle Wu, et al. 2025. OnePiece: Bringing Context Engineering and Reasoning to Industrial Cascade Ranking System. *arXiv preprint arXiv:2509.18091* (2025).
- [4] Ruining He, Lukasz Heldt, Lichan Hong, Raghunandan Keshavan, Shifan Mao, Nikhil Mehta, Zhengyang Su, Alicia Tsai, Yueqi Wang, Shao-Chuan Wang, et al. 2025. Plum: Adapting pre-trained language models for industrial-scale generative recommendations. *arXiv preprint arXiv:2510.07784* (2025).
- [5] Peiyu Hu, Wayne Lu, and Jia Wang. 2026. From ids to semantics: A generative framework for cross-domain recommendation with adaptive semantic tokenization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 14874–14882.
- [6] Yanhua Huang, Yuqi Chen, Xiong Cao, Rui Yang, Mingliang Qi, Yinghao Zhu, Qingchang Han, Yaowei Liu, Zhaoyi Liu, Xuefeng Yao, et al. 2025. Towards large-scale generative ranking. *arXiv preprint arXiv:2505.04180* (2025).
- [7] Yuchen Jiang, Jie Zhu, Xintian Han, Hui Lu, Kunmin Bai, Mingyu Yang, Shikang Wu, Ruihao Zhang, Wenlin Zhao, Shipeng Bai, et al. 2026. TokenMixer-Large: Scaling Up Large Ranking Models in Industrial Recommenders. *arXiv preprint arXiv:2602.06563* (2026).
- [8] Qingyun Liu, Bo Yan, Yang Liu, Yuji Roh, Ekansh Sharma, Likang Yin, Emma Olowo, Min huan Tsai, Yuxuan Li, Diego Uribe, Saksham Aggarwal, Siqi Wu, Yuan Hao, Vikas Kedigehalli, Lukasz Heldt, Lichan Hong, Li Wei, and Xinyang Yi. 2026. TokenMinds: Pretrained User Tokens and Embeddings for User Understanding in Large Recommender Systems. *arXiv:2606.25147* [cs.IR] <https://arxiv.org/abs/2606.25147>
- [9] Zheqi Lv, Tianyu Zhan, Wenjie Wang, Xinyu Lin, Shengyu Zhang, Wenqiao Zhang, Jiwei Li, Kun Kuang, and Fei Wu. 2025. Collaboration of Large Language Models and Small Recommendation Models for Device-Cloud Recommendation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1* (Toronto ON, Canada) (*KDD '25*). Association for Computing Machinery, New York, NY, USA, 962–973. doi:10.1145/3690624.3709335
- [10] Haohao Qu, Wenqi Fan, Zihuai Zhao, and Qing Li. 2025. Tokenrec: Learning to tokenize id for llm-based generative recommendations. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [11] Zihua Si, Zhongxiang Sun, Jiale Chen, Guozhang Chen, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, Jun Xu, and Kun Gai. 2024. Generative retrieval with semantic tree-structured identifiers and contrastive learning. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 154–163.
- [12] Anima Singh, Trung Vu, Nikhil Mehta, Raghunandan Keshavan, Maheswaran Sathiamoorthy, Yilin Zheng, Lichan Hong, Lukasz Heldt, Li Wei, Devansh Tandon, Ed Chi, and Xinyang Yi. 2024. Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems (RecSys '24)*. Association for Computing Machinery, New York, NY, USA.
- [13] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *International Conference on Machine Learning (ICML)*.
- [14] Qiang Zhang, Hanchao Yu, Ivan Ji, Chen Yuan, Yi Zhang, Chihuang Liu, Xiaolong Wang, Christopher E Lambert, Ren Chen, Chen Kovacs, et al. 2025. Efficient Sequential Recommendation for Long Term User Interest Via Personalization. In *2025 IEEE International Conference on Data Mining (ICDM)*. IEEE, 913–922.
- [15] Yang Zhang, Fuli Feng, Jizhi Zhang, Keqin Bao, Qifan Wang, and Xiangnan He. 2025. CoLLM: Integrating Collaborative Embeddings Into Large Language Models for Recommendation. *IEEE Transactions on Knowledge and Data Engineering* 37, 5 (2025), 2329–2340. doi:10.1109/TKDE.2025.3540912
- [16] Da-Wei Zhou, Hai-Long Sun, Jingyi Ning, Han-Jia Ye, and De-Chuan Zhan. 2024. Continual learning with pre-trained models: A survey. *arXiv preprint arXiv:2401.16386* (2024).
- [17] Guorui Zhou, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Shiyao Wang, et al. 2025. OneRec Technical Report. *arXiv preprint arXiv:2506.13695* (2025).
- [18] Jingyi Zhou, Cheng Chen, Kai Zuo, Manjie Xu, Zhendong Fu, Yibo Chen, Xu Tang, and Yao Hu. 2025. HyMiRec: A Hybrid Multi-interest Learning Framework for LLM-based Sequential Recommendation. *arXiv preprint arXiv:2510.13738* (2025).
- [19] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, et al. 2025. Rankmixer: Scaling up ranking models in industrial recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6309–6316.
- [20] Yanyan Zou, Junbo Qi, Lunsong Huang, Yu Li, Kewei Xu, Jiabao Gao, Binglei Zhao, Xuanhua Yu, Sulong Xu, and Shengjie Li. 2026. GenRec: A Preference-Oriented

Generative Framework for Large-Scale Recommendation. In *SIRIG*.

A Soft Tokens Attention Visualization

To better understand the behavioral differences between models utilizing soft tokens and those using textual SIDs (similarly as [4]), we analyze and compare their internal attention patterns. Specifically, we visualize the attention distributions across all transformer layers and attention heads for both options in the context of ranking task.



Figure 7: Max attention scores across all layers/heads for Soft token inputs



Figure 8: Mean attention scores across all layers/heads for Soft token inputs

Figures 7 and 9 present the maximum attention scores observed across all layers and heads for the soft token and textual SID inputs in the user watch history (note that in the textual SID configuration, two `<emb>` tokens are appended to each SID to represent the channel and certain dense features). In these heatmaps, higher attention scores are indicated by higher color intensity.

In the textual SID model (Figure 9), we observe that certain frequent tokens, such as *A1909*, which commonly appears as the leading sub-token of the SID sequence receive very little attention across the interaction history. More importantly, nearly half of the textual SID tokens in the prompt are virtually unattended, appearing as inactive (white) regions. In contrast, the soft token model (Figure 7) shows a non-trivial level of attention across all soft tokens (visualized as varying shades of red). This indicates that by compressing heterogeneous features into soft tokens, every token is leveraged by at least one specialized sub-network or attention head, resulting in a more efficient and comprehensive utilization of the model’s capacity.

Figures 8 and 10 illustrate the mean attention scores averaged across all layers and heads. In the textual SID configuration (Figure 10), the attention is highly sparse and concentrated on a very

G959<emb><emb> A1909 B1130 C948 D937 E845 F1134 G129<emb><emb> A1909 B553 C846 D1236 E1595 F1181
G599<emb><emb> A1909 B208 C1412 D956 E201 F1767 G1693<emb><emb> A1909 B2038 C602 D153 E1575 F704
G2005<emb><emb> A1909 B1592 C953 D1104 E1665 F109 G1112<emb><emb> A1909 B1592 C953 D1104 E1665 F109
G1112<emb><emb> A110 B1084 C756 D1544 E245 F2000 G537<emb><emb> A110 B1084 C456 D1377 E1938 F1925
G213<emb><emb> A84 B1084 C1702 D502 E211 F889 G1897<emb><emb> A1909 B998 C631 D1805 E934 F379
G570<emb><emb> A1218 B1084 C371 D2006 E1409 F1721 G1570<emb><emb> A914 B1084 C2010 D627 E175 F1626
G1308<emb><emb> A1218 B1084 C371 D2006 E1409 F1721 G1570<emb><emb> A1909 B1667 C1581 D504 E110 F814
G1564<emb><emb> A1218 B1084 C99 D2039 E1409 F1887 G1692<emb><emb> A1218 B1084 C557 D1763 E1212 F1221
G537<emb><emb> A1909 B745 C290 D1919 E1853 F297 G1693<emb><emb> A1909 B416 C598 D1654 E1394 F1178
G1352<emb><emb> A1909 B416 C598 D1575 E1394 F794 G753<emb><emb> A1909 B416 C598 D1210 E1394 F1109
G204<emb><emb> A1909 B667 C1756 D1532 E235 F755 G1605<emb><emb> A1909 B667 C1756 D1532 E235 F755
G1605<emb><emb> A1909 B57 C494 D699 E474 F1610 G275<emb><emb> A1218 B1084 C67 D992 E829 F600
G1746<emb><emb> A1909 B2006 C215 D848 E1052 F2038 G1214<emb><emb> A1909 B1667 C1910 D504 E967 F1392
G363<emb><emb> A1909 B208 C996 D1636 E814 F1460 G1410<emb><emb> A1909 B1634 C1558 D184 E1938 F920
G1422<emb><emb> A1513 B516 C1890 D987 E1111 F1855 G537<emb><emb> A1909 B1443 C1846 D741 E1268 F59
G716<emb><emb> A1909 B1366 C1279 D1802 E1683 F14 G973<emb><emb> A1909 B1375 C744 D638 E1677 F489
G624<emb><emb> A1909 B1130 C1293 D1596 E1693 F1042 G1379<emb><emb> A22 B1803 C1949 D1664 E353 F117
G187<emb><emb> A1909 B1366 C1279 D1458 E474 F127 G1698<emb><emb> A1909 B1366 C1279 D261 E985 F351
G513<emb><emb> A1909 B1366 C1279 D1458 E539 F1578 G1698<emb><emb> A1909 B1366 C1279 D769 E474 F1129
G2018<emb><emb> A1909 B1366 C1279 D1458 E474 F1641 G1698<emb><emb> A1909 B208 C1172 D1492 E173 F573
G393<emb><emb> A1909 B1034 C813 D324 E823 F664 G1547<emb><emb> A1218 B1084 C1145 D452 E1405 F920
G741<emb><emb> A1909 B2038 C277 D1114 E1003 F52 G2033<emb><emb>...<end_of_watch_history>

Figure 9: Max attention scores across all layers/heads for textual SID inputs

G959<emb><emb> A1909 B1130 C948 D937 E845 F1134 G129<emb><emb> A1909 B553 C846 D1236 E1595 F1181
G599<emb><emb> A1909 B208 C1412 D956 E201 F1767 G1693<emb><emb> A1909 B2038 C602 D153 E1575 F704
G2005<emb><emb> A1909 B1592 C953 D1104 E1665 F109 G1112<emb><emb> A1909 B1592 C953 D1104 E1665 F109
G1112<emb><emb> A110 B1084 C756 D1544 E245 F2000 G537<emb><emb> A110 B1084 C456 D1377 E1938 F1925
G213<emb><emb> A84 B1084 C1702 D502 E211 F889 G1897<emb><emb> A1909 B998 C631 D1805 E934 F379
G570<emb><emb> A1218 B1084 C371 D2006 E1409 F1721 G1570<emb><emb> A914 B1084 C2010 D627 E175 F1626
G1308<emb><emb> A1218 B1084 C371 D2006 E1409 F1721 G1570<emb><emb> A1909 B1667 C1581 D504 E110 F814
G1564<emb><emb> A1218 B1084 C99 D2039 E1409 F1887 G1692<emb><emb> A1218 B1084 C557 D1763 E1212 F1221
G537<emb><emb> A1909 B745 C290 D1919 E1853 F297 G1693<emb><emb> A1909 B416 C598 D1654 E1394 F1178
G1352<emb><emb> A1909 B416 C598 D1575 E1394 F794 G753<emb><emb> A1909 B416 C598 D1210 E1394 F1109
G204<emb><emb> A1909 B667 C1756 D1532 E235 F755 G1605<emb><emb> A1909 B667 C1756 D1532 E235 F755
G1605<emb><emb> A1909 B57 C494 D699 E474 F1610 G275<emb><emb> A1218 B1084 C67 D992 E829 F600
G1746<emb><emb> A1909 B2006 C215 D848 E1052 F2038 G1214<emb><emb> A1909 B1667 C1910 D504 E967 F1392
G363<emb><emb> A1909 B208 C996 D1636 E814 F1460 G1410<emb><emb> A1909 B1634 C1558 D184 E1938 F920
G1422<emb><emb> A1513 B516 C1890 D987 E1111 F1855 G537<emb><emb> A1909 B1443 C1846 D741 E1268 F59
G716<emb><emb> A1909 B1366 C1279 D1802 E1683 F14 G973<emb><emb> A1909 B1375 C744 D638 E1677 F489
G624<emb><emb> A1909 B1130 C1293 D1596 E1693 F1042 G1379<emb><emb> A22 B1803 C1949 D1664 E353 F117
G187<emb><emb> A1909 B1366 C1279 D1458 E474 F127 G1698<emb><emb> A1909 B1366 C1279 D261 E985 F351
G513<emb><emb> A1909 B1366 C1279 D1458 E539 F1578 G1698<emb><emb> A1909 B1366 C1279 D769 E474 F1129
G2018<emb><emb> A1909 B1366 C1279 D1458 E474 F1641 G1698<emb><emb> A1909 B208 C1172 D1492 E173 F573
G393<emb><emb> A1909 B1034 C813 D324 E823 F664 G1547<emb><emb> A1218 B1084 C1145 D452 E1405 F920
G741<emb><emb> A1909 B2038 C277 D1114 E1003 F52 G2033<emb><emb>...<end_of_watch_history>

Figure 10: Mean attention scores across all layers/heads for textual SID inputs

small subset of tokens. This concentration suggests substantial representation redundancy, with the majority of layers and heads focusing on the same few tokens. Conversely, the mean attention map for the soft token model (Figure 8) is significantly denser and more distributed. This indicates that the attention mass is spread across the sequence, allowing different attention heads and layers to capture complementary signals from the input.

In summary, these visual analyses demonstrate that the soft token representations generated by Token Factory are not only computationally efficient due to prompt compression, but they are also utilized more effectively and balanced by the attention mechanism, successfully mitigating attention redundancy.